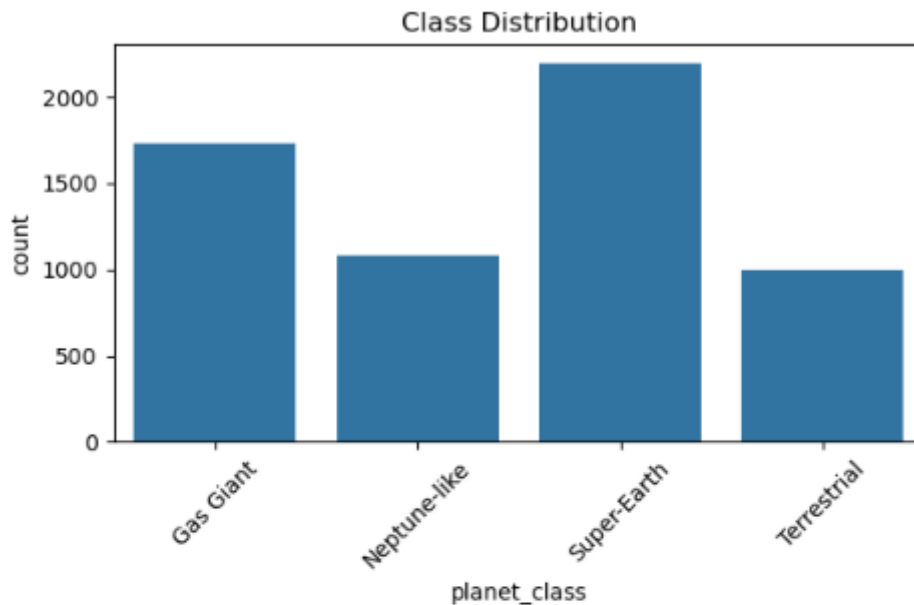


Exoplanet classification with Support Vector Machines and Decision Trees

Q1. Explore the exoplanet classes

Examine the distribution of the planet classes. Are they balanced? How could class imbalances affect classification performance? Explore the selected variables and describe their individual distributions. Identify which classes appear similar based on these variables. How might such similarities influence classification models?



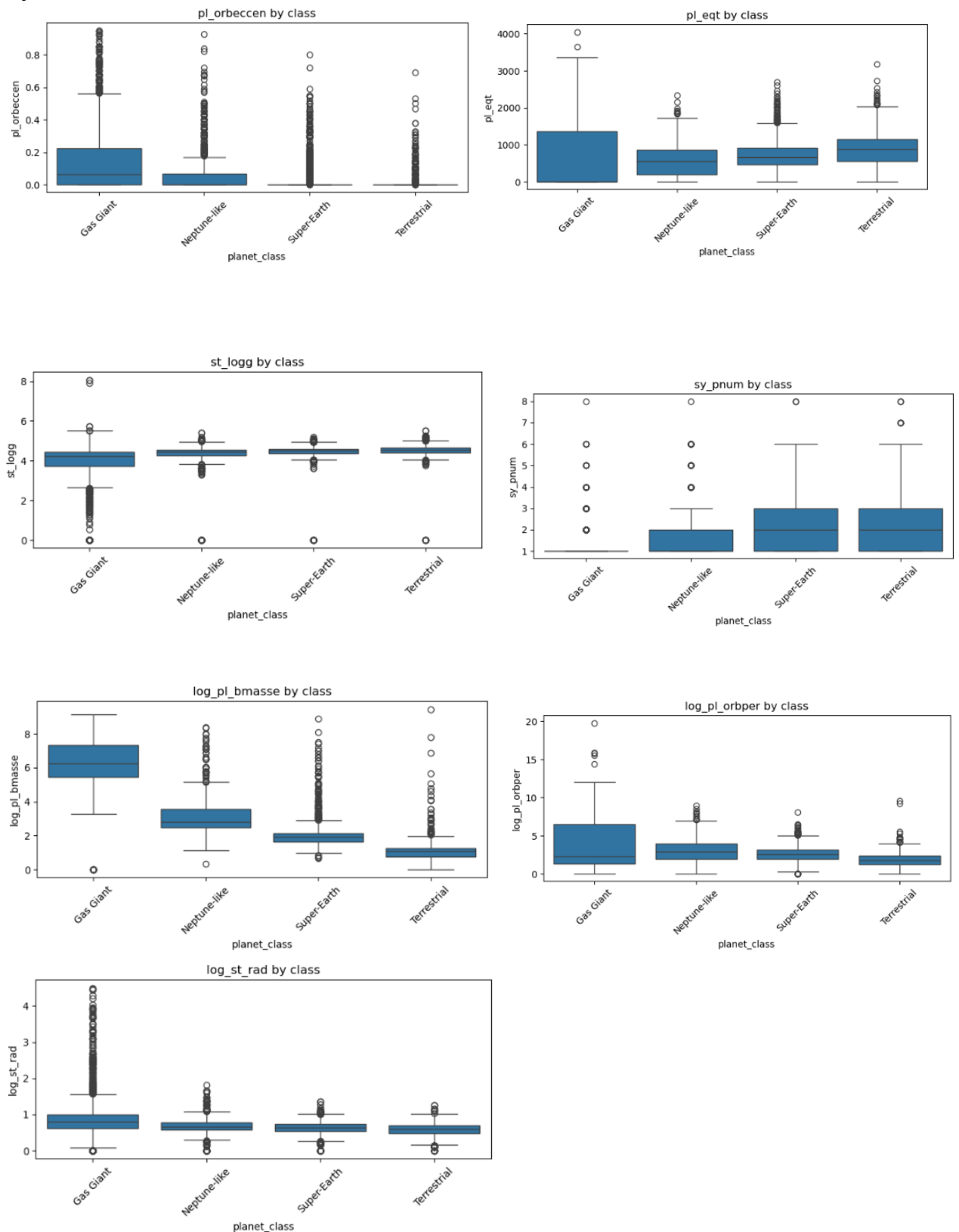
```
Class counts:
planet_class
Super-Earth    2195
Gas Giant      1734
Neptune-like   1079
Terrestrial     996
Name: count, dtype: int64
```

```
Class percentages (%):
planet_class
Super-Earth    36.56
Gas Giant      28.88
Neptune-like   17.97
Terrestrial    16.59
Name: proportion, dtype: float64
```

Super Earth class is dominating, it occupies more than third of data, lowest percentage is Terrestrial class. Classes are not perfectly balanced but it is not severely imbalanced. The difference is not that big that a model would ignore those classes.

How I selected my features: firstly I looked through every features distribution and their log transformation's distribution, if atleast 3 out of 4 classes had very similar distribution, I put that away from my features list. Then I was left with 7 features and if log transformation did not have any effect I just did not complete the transformation.

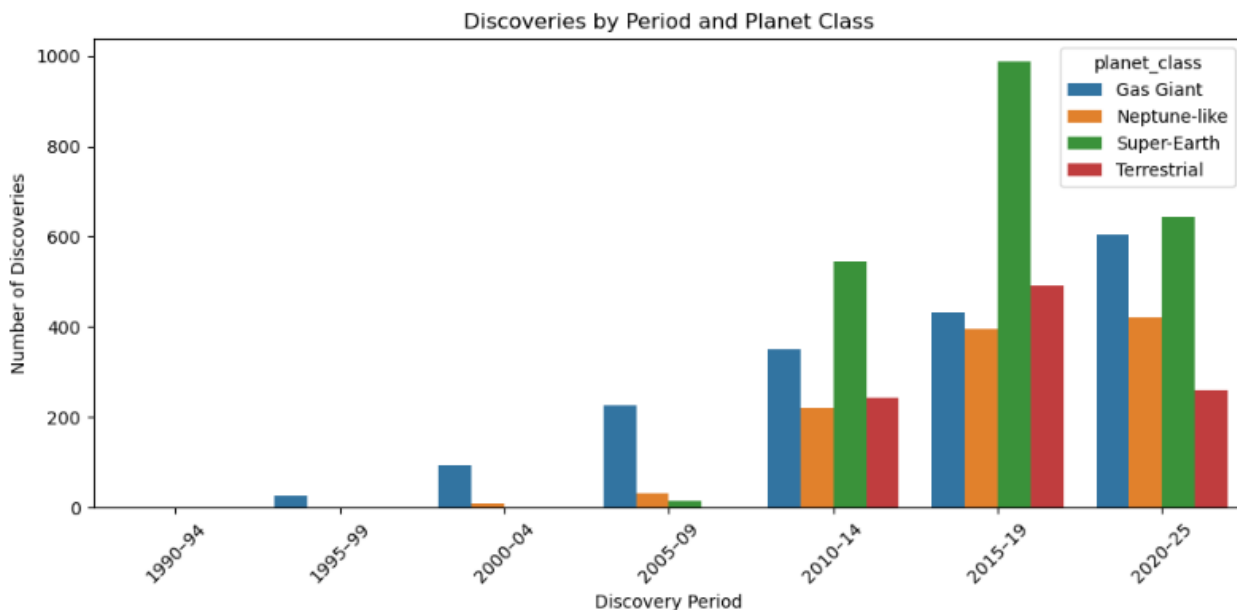
My selected features and their distributions:



Super-Earth and Terrestrial classes looks similar in these variables: `log_st_rad`; `st_logg`; `pl_orbeccen`; `sy_pnum`. These similarities could influence false-positive and false-negative numbers. Gas giant and Neptune-like classes looks fairly distinctive.

Q2. Investigate exoplanet discovery patterns over time

Are there any noticeable patterns in the discovery years of exoplanets? You can group years into broader periods and explore how other variables behave across these periods. No modeling is required here - focus on descriptive insights.



We can see that until 2010 there were none or little discoveries of Neptune-like, Super-Earth or Terrestrial classes. The biggest contributor for mass discoveries of these classes is Kepler space telescope which was first NASA's planet-hunting mission launched on 2009 March 6.

Q3. Explore which features are the most important in classification of exoplanets

Train a Support Vector Machine with a linear kernel. Compare model performance with and without scaling the features. How does scaling affect the results? Why is feature scaling generally considered mandatory for distance-based models like SVM, but less critical for non-distance-based models like Decision Trees? Identify the most influential variables for predicting classes. Visualize how these variables differ across classes. Do these features make sense physically (e.g., orbital radius, temperature)?

I tried three different models: first is with my 7 features, second is with only those top 3 features I got from my first model, and third model was only with `pl_bmasse` feature. All of my models were tried with scaling and without scaling.

First model with 7 features

=== SVM WITHOUT SCALING ===

Accuracy: 0.8714

	precision	recall	f1-score	support
Gas Giant	0.93	0.95	0.94	433
Neptune-like	0.80	0.64	0.71	270
Super-Earth	0.83	0.91	0.87	549
Terrestrial	0.93	0.89	0.91	249
accuracy			0.87	1501
macro avg	0.87	0.85	0.86	1501
weighted avg	0.87	0.87	0.87	1501

=== SVM WITH SCALING ===

Accuracy: 0.8681

	precision	recall	f1-score	support
Gas Giant	0.94	0.95	0.95	433
Neptune-like	0.79	0.64	0.71	270
Super-Earth	0.82	0.91	0.86	549
Terrestrial	0.93	0.88	0.90	249
accuracy			0.87	1501
macro avg	0.87	0.85	0.86	1501
weighted avg	0.87	0.87	0.87	1501

Approximate feature importance (mean |coef| across classes):

log_pl_bmasse	3.924948
log_pl_orbper	0.270215
log_st_rad	0.259982
sy_pnum	0.233009
pl_eqt	0.127842
st_logg	0.098835
pl_orbeccen	0.072788

My prediction that Super-Earth and Terrestrial would be the most difficult to distinguish was not correct. In fact it was Neptune-like because it has only 0,64 recall, that means it missed a lot of true positive results while being compared to other classes. While it is important to scale our data, in this example it did not have that big of a result, the reason might be that I transformed some of my selected variables. Nonetheless it is very important to scale our results, since in SVM margin is being calculated, and if for example one variable is in the range of [0;1] and the other is in [-100000;10000], the latter feature would have the biggest impact in that calculation and the first one would have a very small effect. Data tree does not really need scaling since it is based on threshold. The top 3 features are logical, since every class in mass feature are distinguished from each other, also other feature have small importance because looking into their distribution, at least two classes are more or less the same.

Second model with top 3 features

=== SVM WITHOUT SCALING (3 features) ===

Accuracy: 0.8674

	precision	recall	f1-score	support
Gas Giant	0.93	0.96	0.94	433
Neptune-like	0.80	0.63	0.71	270
Super-Earth	0.82	0.91	0.86	549
Terrestrial	0.93	0.88	0.91	249
accuracy			0.87	1501
macro avg	0.87	0.84	0.85	1501
weighted avg	0.87	0.87	0.86	1501

=== SVM WITH SCALING (3 features) ===

Accuracy: 0.8674

	precision	recall	f1-score	support
Gas Giant	0.93	0.96	0.94	433
Neptune-like	0.80	0.63	0.70	270
Super-Earth	0.82	0.91	0.86	549
Terrestrial	0.94	0.88	0.91	249
accuracy			0.87	1501
macro avg	0.87	0.84	0.85	1501
weighted avg	0.87	0.87	0.86	1501

The results are very similar to the first model, this time scaling did not have any impact, might be because all of the features were transformed.

Third model with only pl_bamsse

```

-----
TESTING FEATURE: log_pl_bamsse | scaling=False
=====
Accuracy: 0.8681
      precision    recall  f1-score   support

   Gas Giant      0.93      0.96      0.95        433
 Neptune-like      0.79      0.58      0.67        270
 Super-Earth      0.81      0.93      0.86        549
 Terrestrial      0.97      0.90      0.93        249

   accuracy
 macro avg      0.88      0.84      0.85        1501
weighted avg      0.87      0.87      0.86        1501

=====
TESTING FEATURE: log_pl_bamsse | scaling=True
=====
Accuracy: 0.8681
      precision    recall  f1-score   support

   Gas Giant      0.93      0.96      0.95        433
 Neptune-like      0.80      0.57      0.67        270
 Super-Earth      0.81      0.93      0.86        549
 Terrestrial      0.97      0.90      0.93        249

   accuracy
 macro avg      0.88      0.84      0.85        1501
weighted avg      0.87      0.87      0.86        1501

```

This model has the same results as the first model with scalling.

Q4. Model complexity and support vectors

Train nonlinear SVM (e.g., using the radial basis function kernel) with different values of C and gamma. For each model, evaluate the training accuracy, test accuracy, and the number of support vectors. How do changes in C or gamma influence generalization and overfitting? By varying these hyperparameters, illustrate examples of underfitting, proper fitting, and overfitting. Discuss the observed results, particularly the relationships between margin width, support vector count, and model flexibility.

```

=== UNDERFITTING EXAMPLES ===
      C      gamma  train_acc  test_acc  support_vectors
0  0.01  0.010000  0.523641  0.519149           4077
1  0.01  0.027826  0.616076  0.616312           3783
2  0.01  0.077426  0.632151  0.633333           3631
3  0.01  0.215443  0.628369  0.629078           3592
4  0.01  0.599484  0.616785  0.618440           3796

=== PROPER FITTING EXAMPLES ===
      C      gamma  train_acc  test_acc  support_vectors
42  1.668101  0.077426  0.878014  0.860284           2068
51  5.994843  0.027826  0.877305  0.865957           1989
52  5.994843  0.077426  0.893617  0.870922           1802
60  21.544347  0.010000  0.873759  0.863121           1961
61  21.544347  0.027826  0.887943  0.871631           1773

=== OVERFITTING EXAMPLES ===
      C      gamma  train_acc  test_acc  support_vectors
46  1.668101  4.641589  0.968794  0.794326           3138
47  1.668101  12.915497  0.988416  0.718440           3638
48  1.668101  35.938137  0.997636  0.641844           4045
49  1.668101  100.000000  0.999764  0.422695           4210
55  5.994843  1.668101  0.964775  0.834752           2410

=== BEST CONFIGURATION ===
C      1000.000000
gamma      0.010000
train_acc      0.901182
test_acc      0.881560

```

Underfitting models uses low C, gamma values, have a lot of support vectors (from 3500 to 4100) which should mean it has wide margin and the models are too simple to distinguish classes. When hyperparameter C is low it allows more misclassifications because it does not give a penalty for misclassification and low gamma does not give enough curve to our upper, lower and desision boundaries. This results in poor test and training accuracy.

Overfitting models uses moderate to high C and high gamma values and have a lot of support vectors (from 2500 to 4000) which should mean it has very narrow margin and the models have hard time generalizing the data. moderate C values leaves less room for misclassifications because it gives a bigger penalty and high gamma values curves up upper, lower and decision boundaries which I think mean our model remembers noise too much. This results in very high training accuracy and poor test accuracy.

Proper fitting models uses moderate to high C values and low gamma values and have moderate support vectors (1800 to 2100) which should mean it has narrower margin. Moderate and high C values gives bigger penalty for misclassifications and low gamma values ensures that the curve won't be too complex. This results in good test and training accuracy.

Q5. Analyze exoplanet classification rules

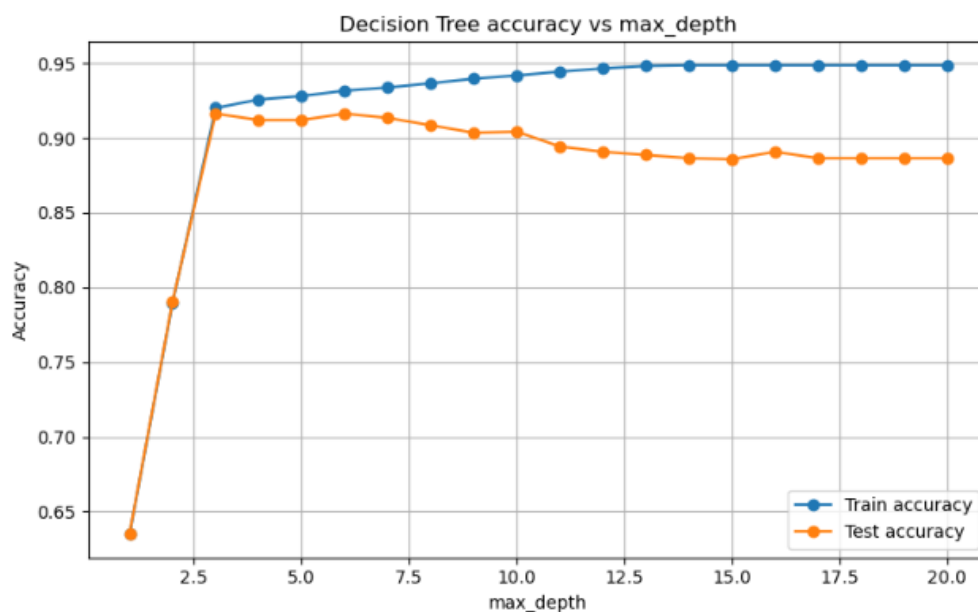
Train a Decision Tree classifier. Start with a large depth. Observe accuracy on train/test sets. Apply complexity-based pruning and early stopping (e.g. limit max_depth, min_samples_leaf, or use ccp_alpha). How do pruning and early stopping affect model performance and complexity? Plot tree accuracy vs. pruning parameter. Visualize the pruned tree (or first few levels). Extract several learned decision rules. Compute and visualize feature importance from the Decision Tree. Are the most important features consistent with the SVM results? Do they appear physically logical?

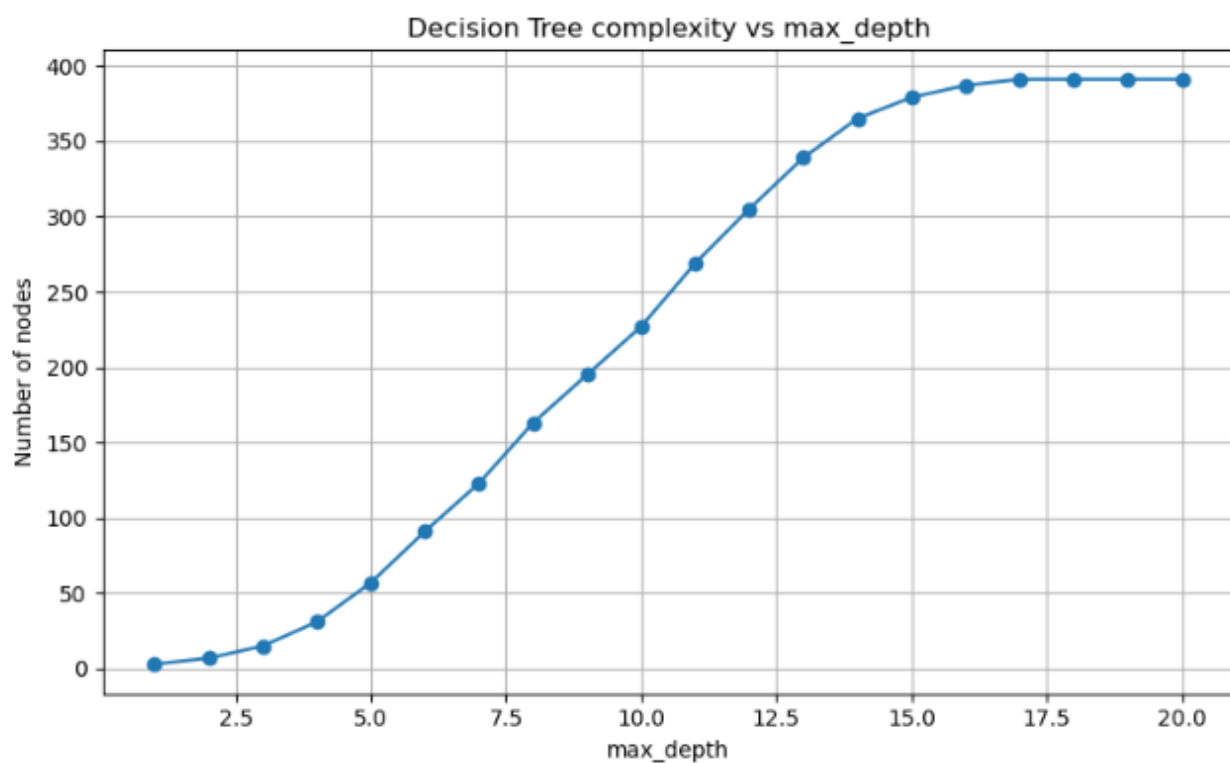
Train accuracy: 1.0

Test accuracy: 0.8787234042553191

=== Accuracy vs max_depth (first rows) ===

	max_depth	train_acc	test_acc	nodes
0	1	0.634988	0.635461	3
1	2	0.790071	0.790780	7
2	3	0.920095	0.916312	15
3	4	0.925768	0.912057	31
4	5	0.928132	0.912057	57

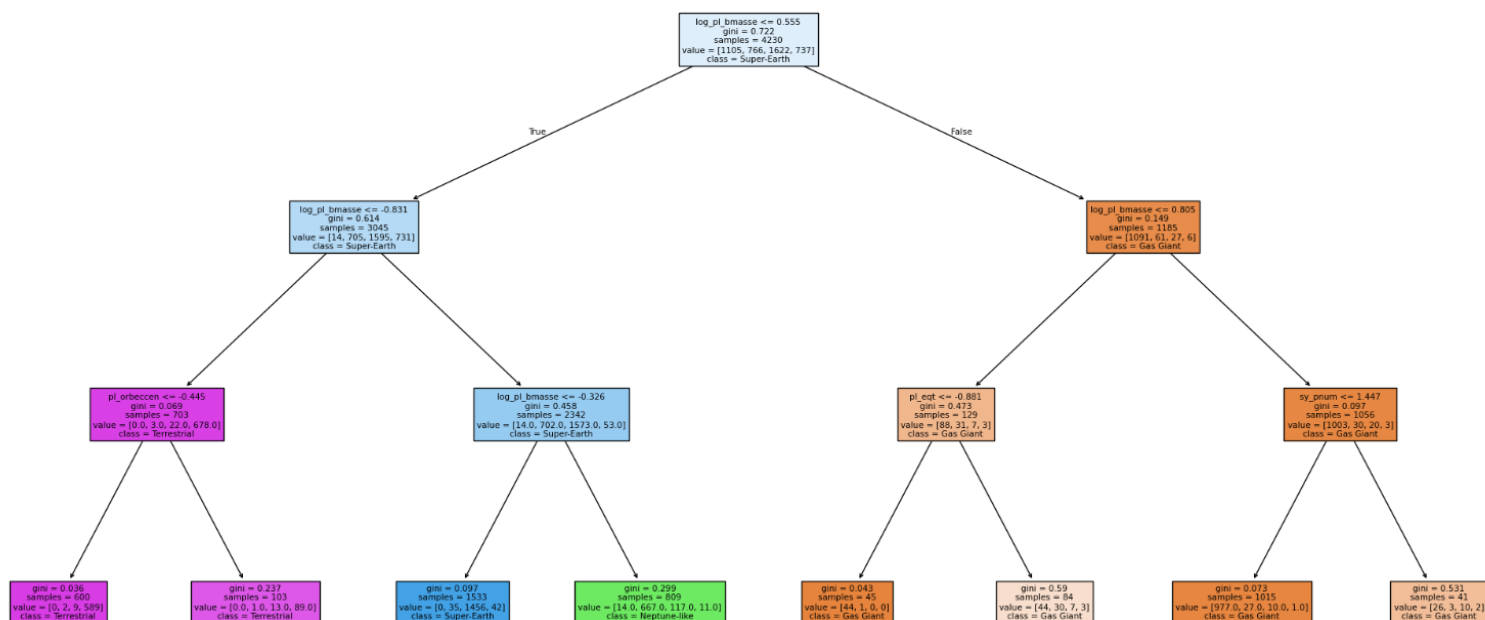




```

=== Best depth according to test accuracy ===
Best max_depth: 3
Train accuracy at best depth: 0.9200945626477541
Test accuracy at best depth: 0.9163120567375886
  
```

Decision Tree (top levels), max_depth=3, min_samples_leaf=5



```

=== Decision rules (truncated) ===
|--- log_pl_bmasse <= 0.56
| |--- log_pl_bmasse <= -0.83
| | |--- pl_orbeccen <= -0.45
| | | |--- class: Terrestrial
| | |--- pl_orbeccen > -0.45
| | | |--- class: Terrestrial
| |--- log_pl_bmasse > -0.83
| | |--- log_pl_bmasse <= -0.33
| | | |--- class: Super-Earth
| | |--- log_pl_bmasse > -0.33
| | | |--- class: Neptune-like
|--- log_pl_bmasse > 0.56
| |--- log_pl_bmasse <= 0.81
| | |--- pl_eqt <= -0.88
| | | |--- class: Gas Giant
| | |--- pl_eqt > -0.88
| | | |--- class: Gas Giant
| |--- log_pl_bmasse > 0.81
| | |--- sy_pnum <= 1.45
| | | |--- class: Gas Giant
| | |--- sy_pnum > 1.45
| | | |--- class: Gas Giant

=== Feature importances from decision tree ===
log_pl_bmasse    0.992525
pl_eqt           0.003874
sy_pnum          0.002667
pl_orbeccen      0.000934
st_logg          0.000000
log_pl_orbper    0.000000
log_st_rad       0.000000

```

Firstly I trained a model with no limit to max_depth and leaf required only one sample. I got training accuracy 1 and test accuracy 0,88, which indicates that our model is overfitting. Then I trained a model where max_depth was set to 20 and a leaf was required to have atleast 5 samples. We got best results when our depth is between 3-6 and after that the model becomes overfitting. The best model had max_depth 3 and it achieved training accuracy 0,92 and test accuracy 0,916. Regarding the complexity of the model, if max_depth reaches bigger level then five, the number of nodes increases very drastically making the model complex and overfitting. Looking into importances of variables we can clearly see that mass is basically the one parameter that indicates a class. Looking into the tree, the first and the second split uses mass feature which means it has the biggest impact in separating the classes and therefore it is the most important feature in this model.

Q6. Find the outlier exoplanets

Use a method of your choice to detect outliers (e.g., z-score, IQR, or visual inspection). Which planets appear as outliers? Which variables make them unusual? Examine how SVM and DT classify these outliers. Are they often misclassified? If you remove them and retrain, how does overall accuracy change? Reflect on the impact of outliers on both models, which one is more sensitive? Should outliers be excluded or maybe modeled as a separate class?

Detected 475 outliers out of 5640 samples.

```

Class distribution among outliers:
planet_class
Gas Giant      353
Super-Earth    51
Neptune-like   41
Terrestrial    30

```

Since most of the outliers are Gas Giant planets and mass has by far the biggest significance, we can say that mass makes them unusual.

```
=== WITH outliers: overall accuracy (train/test) ===
SVM   train: 0.901, test: 0.882
Tree  train: 0.920, test: 0.916

SVM accuracy on TEST outliers : 0.890
Tree accuracy on TEST outliers : 0.937

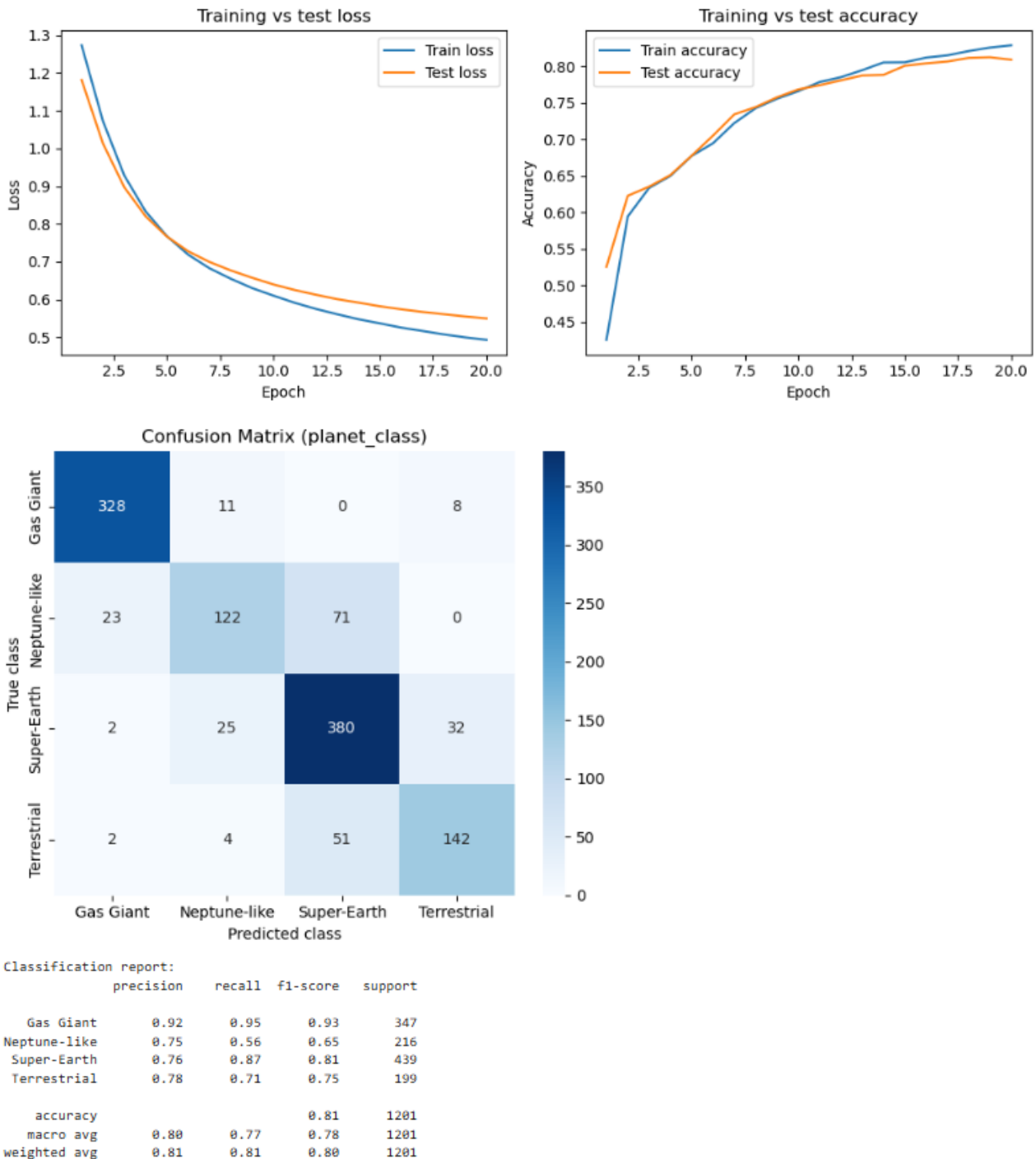
=== WITHOUT outliers: overall accuracy (train/test) ===
SVM   train: 0.903, test: 0.884
Tree  train: 0.916, test: 0.916
```

Both models correctly classifies outliers pretty well, but data tree model classifies them better. But if we retrain the model without outliers our results does not change drastically.

Exoplanet classification with Artificial Neural Networks

Q1. Is there a class of exoplanets that is often confused with the other classes?

Implement a minimal working ANN binary classifier using PyTorch. Suggestion, use 1 hidden layer, start with small number of neurons. Train for about 10 - 20 epochs. Record training and testing accuracy as well as loss for each epoch. Plot learning curves, confusion matrix and discuss the results and errors.



A model with one hidden layer, 16 neurons was trained for 20 epochs. After 20 epochs these were the results:

Epoch 20/20 | train_loss=0.4929, train_acc=0.8291, test_loss=0.5497, test_acc=0.8093

These results does not show significant overfitting or underfitting, although the numbers are not that good. From confusion matrix we can see that the most missclasiffied class is Neptune-like and it mostly missclassified as Super-Earth. It also has a very low 0,56 recall and 0,65 f1-score.

Q2. Fix the misclassification on the model side

q2.1. Compute weights of classes and retrain the model. Check the classification metrics again and discuss them. Experiment further with architecture depth and width. Train networks with different number of hidden-layers (keep all the models, do not overwrite the model if it does not look good). Direct your architectural decisions to aim for higher accuracy and describe what you found worked, why it worked or didn't (e.g. try various options like number of nodes, different optimizers (SGD, Adam), with different learning rates).

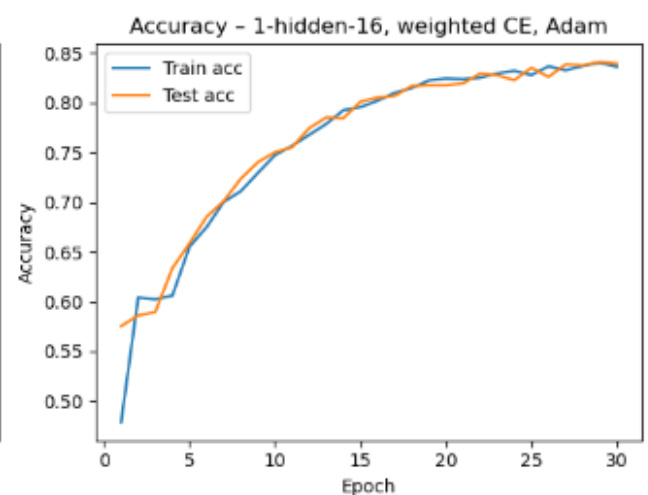
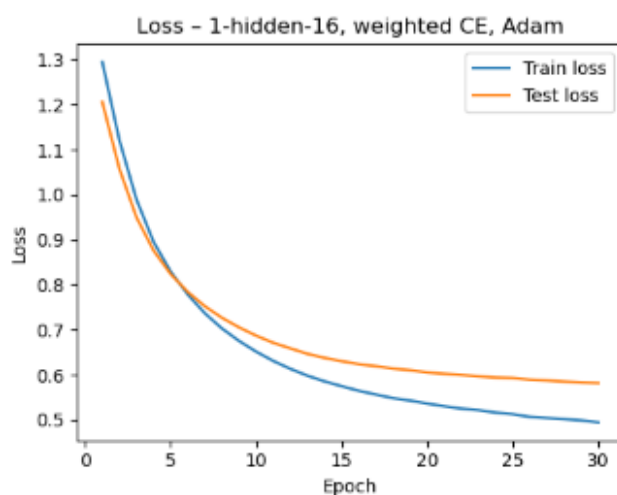
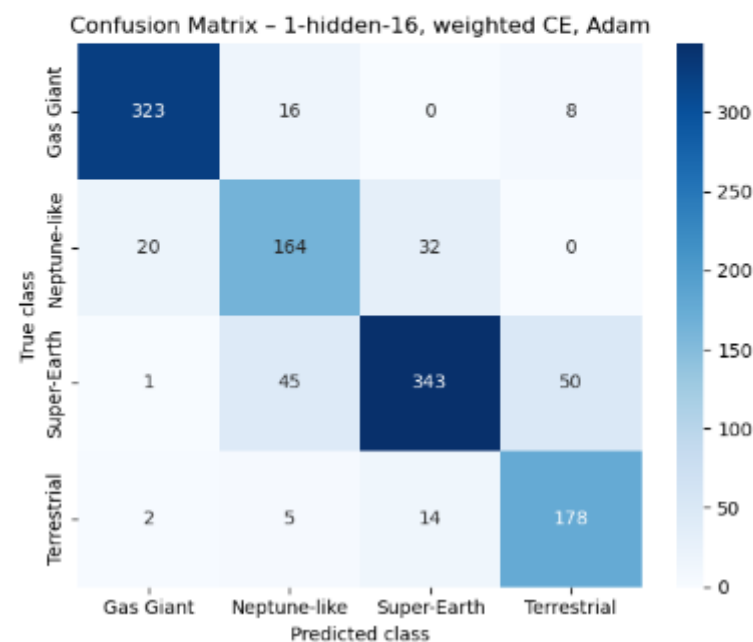
q2.2. Does regularization improve things (dropout, early stopping, L2)?

A model with one hidden layer, 16 neurons, using weights and Adam was trained for 30 epochs. After 30 epochs these were the results:

Epoch 30/30 | train_loss=0.4943, train_acc=0.8361, test_loss=0.5815, test_acc=0.8393

Classification report:

	precision	recall	f1-score	support
Gas Giant	0.93	0.93	0.93	347
Neptune-like	0.71	0.76	0.74	216
Super-Earth	0.88	0.78	0.83	439
Terrestrial	0.75	0.89	0.82	199
accuracy			0.84	1201
macro avg	0.82	0.84	0.83	1201
weighted avg	0.85	0.84	0.84	1201



regularization (weighted, dropout=0.3, L2)

train_loss=0.5364, train_acc=0.7945, test_loss=0.4915, test_acc=0.8618

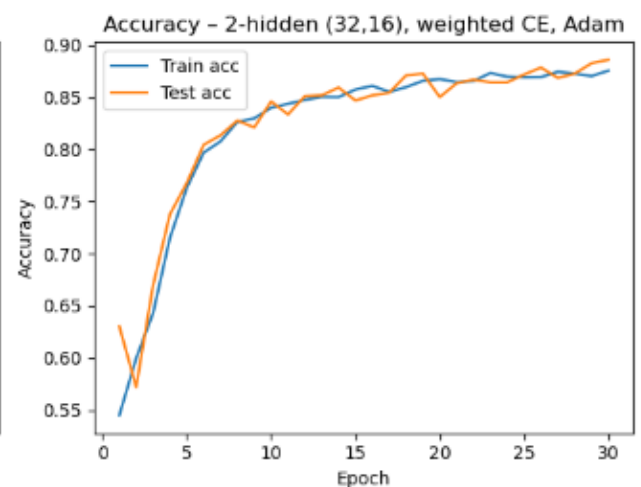
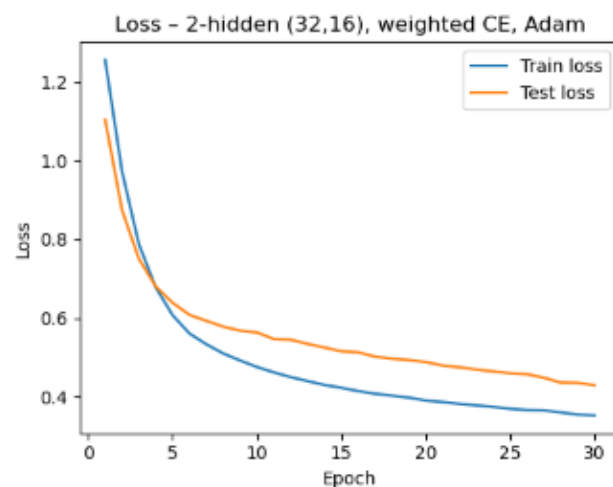
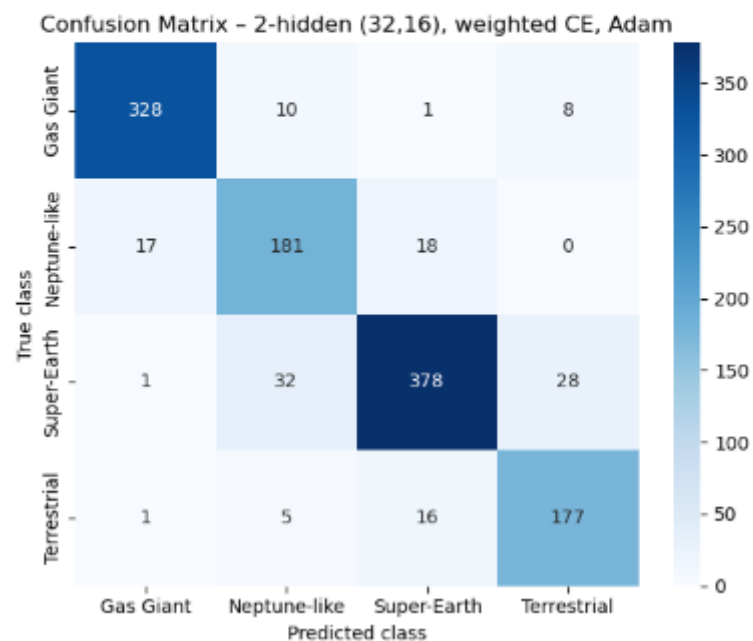
regularization did help

A model with two hidden layers, 32 and 16 neurons, using weights and Adam was trained for 30 epochs. After 30 epochs these were the results:

Epoch 30/30 | train_loss=0.3525, train_acc=0.8755, test_loss=0.4293, test_acc=0.8859

Classification report:

	precision	recall	f1-score	support
Gas Giant	0.95	0.95	0.95	347
Neptune-like	0.79	0.84	0.82	216
Super-Earth	0.92	0.86	0.89	439
Terrestrial	0.83	0.89	0.86	199
accuracy			0.89	1201
macro avg	0.87	0.88	0.88	1201
weighted avg	0.89	0.89	0.89	1201



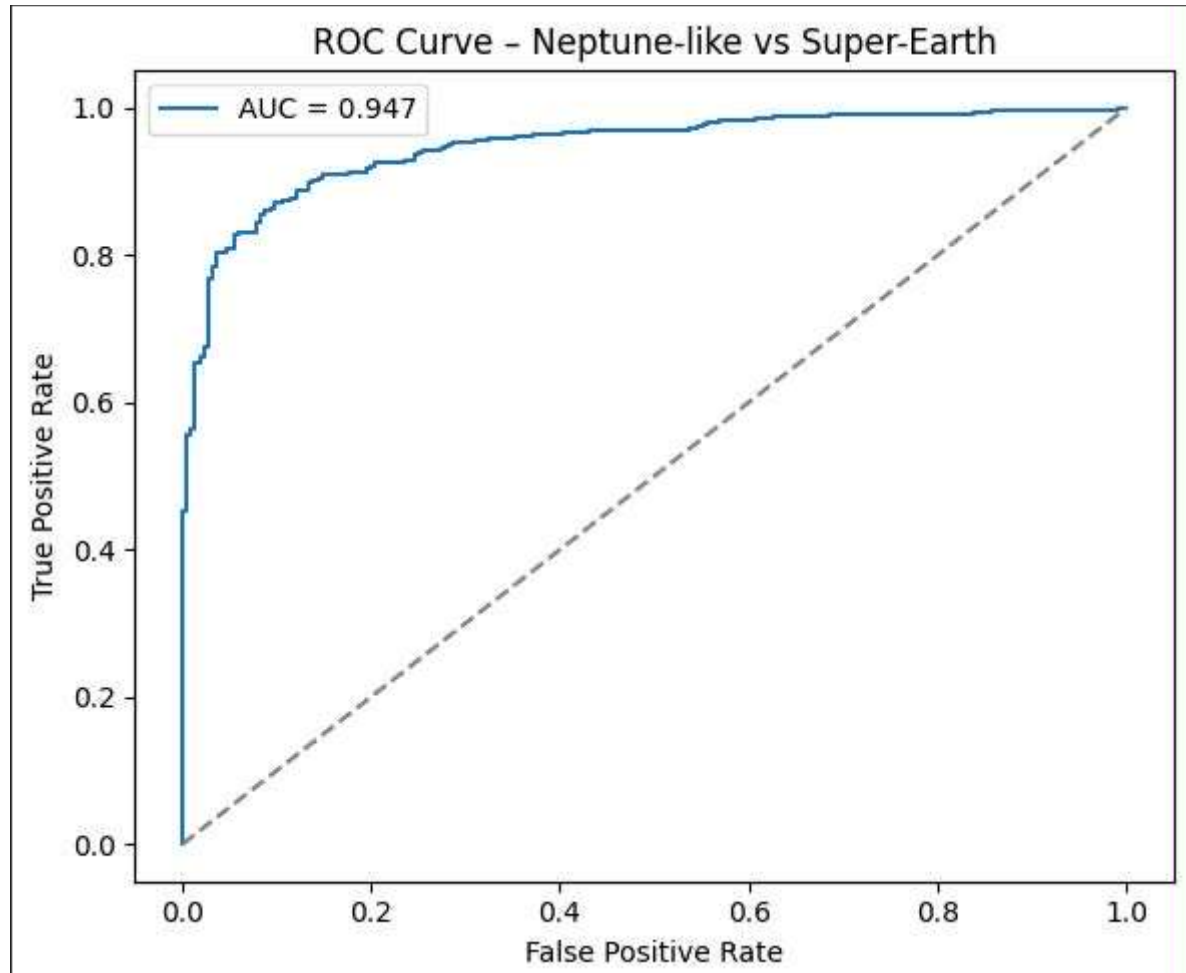
regularization(weighted, dropout=0.3, L2)

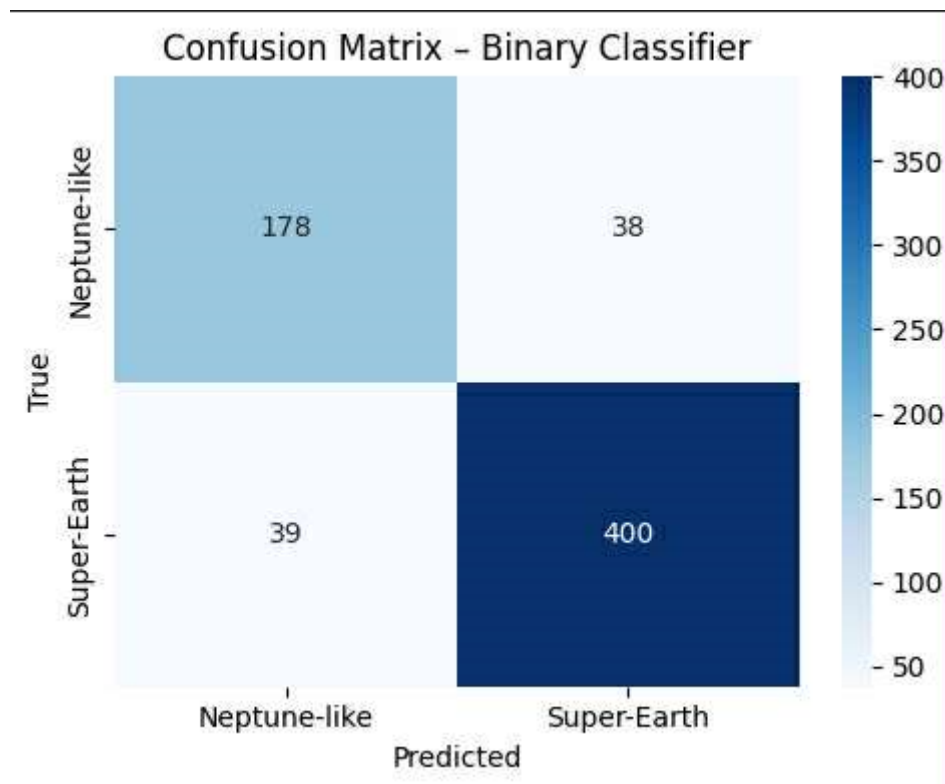
train_loss=0.5005, train_acc=0.8049, test_loss=0.4549, test_acc=0.8668

Regularization did not help

Q3. Fix the misclassification on the data side

Pick the most confused pair or classes from Q1 and build a binary classifier focused on that pair. Tune hyperparameters for this binary task. Report the classification metrics (e.g. precision/recall/F1 and ROC-AUC) for the pair.





	precision	recall	f1-score	support
Neptune-like	0.81	0.86	0.83	216
Super-Earth	0.93	0.90	0.91	439
accuracy	0.89			655
macro avg	0.87	0.88	0.87	655
weighted avg	0.89	0.89	0.89	655

test_acc=0.8870

All metrics looks failry good, especially recall.

AI use

All of the code was writting by AI, comments were written by me (as you can see from mistakes)